

10. Large Language Models (LLM): Tokenization, Inference, and Mini Fine-Tuning

Algorithm Covered:

- **Large Language Models (Transformer Architecture)**

Core Capabilities Explored:

1. **Subword Tokenization:** Token IDs, Attention Masks, Vocabulary Embeddings
2. **Multi-Head Self-Attention Mechanics:** Q , K , V projections and softmax attention weights
3. **Zero-Shot & Downstream Inference:** High-level Hugging Face Pipelines
4. **Text Generation & Decoding Strategies:** Greedy Search vs. Temperature vs. Top-p (Nucleus) Sampling
5. **Mini Fine-Tuning Demonstration:** Sequence classification adaptation using PyTorch and Hugging Face Transformers

Frameworks: Hugging Face `transformers` and `torch`

```
In [1]: # ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- PyTorch: used for the mini fine-tuning demo ----
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset
# ---- Hugging Face Transformers: tokenizers, models, pipelines, generation configs ----
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    AutoModelForCausalLM,
    GenerationConfig,
    pipeline
)

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available
else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['figure.dpi'] = 110

torch.manual_seed(42)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # GPU if available
print(f"PyTorch on {device}")
```

D:\ML\Assignment\.venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html

from .autonotebook import tqdm as notebook_tqdm

PyTorch on cpu

1. Subword Tokenization & Attention Masks

Mathematical Concept:

Language models do not operate on raw characters or entire words; they process **subword tokens** (Byte-Pair Encoding / WordPiece). Given input text S , the tokenizer decomposes S into token IDs $T = [t_1, t_2, \dots, t_L]$ and generates an **Attention Mask** $M \in \{0, 1\}^L$ ensuring padding tokens receive zero attention during self-attention computation:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + (M - 1) \cdot \infty \right) V$$

Step-by-Step Algorithm (Transformer Tokenization)

1. Normalize and split the raw text (lowercase, punctuation handling, etc.).
2. Apply subword segmentation (WordPiece/BPE): frequent words stay whole, rare words split into pieces + "##" continuations.
3. Map each subword to its integer vocabulary ID.
4. Add special tokens ([CLS], [SEP]) and pad/truncate to a fixed length.
5. Build the attention mask: 1 for real tokens, 0 for padding.
6. Feed IDs + mask into the transformer's embedding and self-attention layers.

```
In [2]: # ---- Load a pre-trained DistilBERT tokenizer (fast, subword/WordPiece based) ----
model_id = "distilbert/distilbert-base-uncased-finetuned-sst-2-english"
print(f"Loading Tokenizer: {model_id}...")
tokenizer = AutoTokenizer.from_pretrained(model_id)

sample_text = "Machine learning and deep neural networks are transforming modern science!"
# Tokenize with padding/truncation so the output has a fixed shape (here 20 tokens)
encoded = tokenizer(
    sample_text,
    padding="max_length",
    max_length=20,
    truncation=True,
    return_tensors="pt"
)

# Convert integer IDs back to readable subword tokens
tokens = tokenizer.convert_ids_to_tokens(encoded['input_ids'][0])
print(f"Original Text: {sample_text}\n")
print(f"Subword Tokens ({len(tokens)}):\n", tokens)
print(f"Input IDs:      \n", encoded['input_ids'][0].tolist())
print(f"Attention Mask: \n", encoded['attention_mask'][0].tolist())
```

Loading Tokenizer: distilbert/distilbert-base-uncased-finetuned-sst-2-english...

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Original Text: Machine learning and deep neural networks are transforming modern science!

Subword Tokens (20):

['[CLS]', 'machine', 'learning', 'and', 'deep', 'neural', 'networks', 'are', 'transforming', 'modern', 'science', '!', '[SEP]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]']

Input IDs:

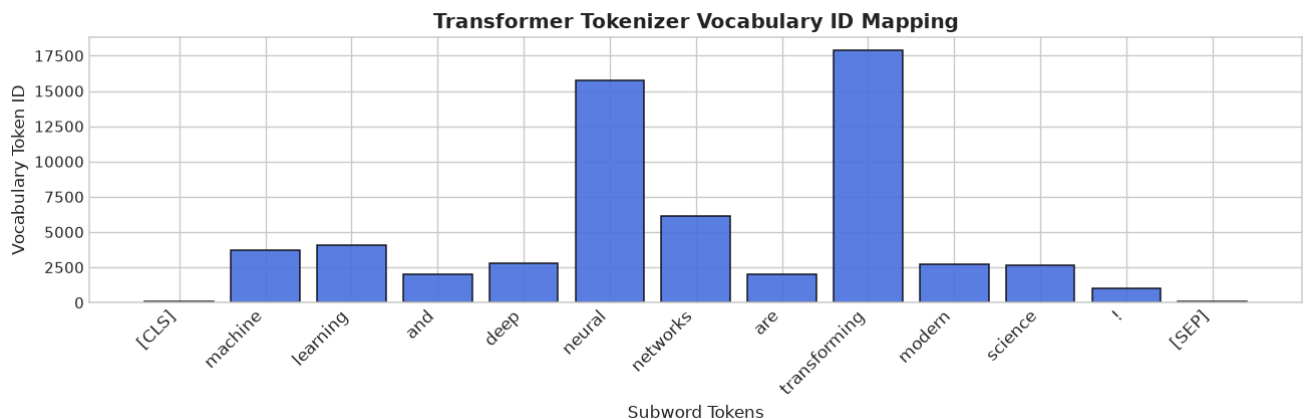
[101, 3698, 4083, 1998, 2784, 15756, 6125, 2024, 17903, 2715, 2671, 999, 102, 0, 0, 0, 0, 0, 0, 0]

Attention Mask:

[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0]

```
In [3]: # ---- Visualize the tokenizer's vocabulary-ID mapping ----
# Keep only real tokens (attention mask = 1) for a clean bar chart.
active_len = int(encoded['attention_mask'].sum().item())
active_tokens = tokens[:active_len]
active_ids = encoded['input_ids'][0][:active_len].tolist()

plt.figure(figsize=(12, 4))
plt.bar(range(len(active_tokens)), active_ids, color='royalblue', edgecolor='k', alpha=0.85)
plt.xticks(range(len(active_tokens)), active_tokens, rotation=45, ha='right', fontsize=11)
plt.title("Transformer Tokenizer Vocabulary ID Mapping", fontsize=13, fontweight='bold')
plt.xlabel("Subword Tokens", fontsize=11)
plt.ylabel("Vocabulary Token ID", fontsize=11)
plt.tight_layout()
plt.show()
```



2. Transformer Inference: Sentiment Classification Pipeline

We load a lightweight, production-grade distilled transformer (`DistilBERT` fine-tuned on the Stanford Sentiment Treebank / SST-2) to compute class logits and calibrated posterior probabilities.

Inference Process (Hugging Face Pipeline)

1. Load the pre-trained checkpoint (weights + tokenizer).
2. Tokenize each input sentence exactly as during pre-training.
3. Run a forward pass through the transformer encoder.
4. Apply softmax to the classification head logits to obtain class probabilities.
5. Return the highest-probability label with its confidence score.

```
In [4]: # ---- Load a pre-trained sentiment-analysis pipeline (DistilBERT SST-2) ----
classifier = pipeline("sentiment-analysis", model=model_id, device=-1) # -1 = run on CPU
```

```

test_sentences = [
    "The new neural network model demonstrated extraordinary accuracy and remarkable speed.",
    "The dataset was poorly labeled, noisy, and the model completely failed to converge.",
    "The gradient descent algorithm performed reasonably well, meeting standard expectations.",
    "A catastrophic failure occurred during training due to exploding gradient issues."
]

# One forward pass per sentence -> label + calibrated confidence score
predictions = classifier(test_sentences)

# Collect results in a table for display
results_table = []
for sent, pred in zip(test_sentences, predictions):
    results_table.append({
        "Sentence": sent,
        "Predicted Label": pred['label'],
        "Confidence Score": f"{pred['score']*100:.2f}%"
    })

df_preds = pd.DataFrame(results_table)
display(df_preds)

```

```

Loading weights:  0%|          | 0/104 [00:00<?, ?it/s]
Loading weights: 83%|██████████| 86/104 [00:00<00:00, 855.44it/s]
Loading weights: 100%|██████████| 104/104 [00:00<00:00, 1004.43it/s]

```

	Sentence	Predicted Label	Confidence Score
0	The new neural network model demonstrated extr...	POSITIVE	99.99%
1	The dataset was poorly labeled, noisy, and the...	NEGATIVE	99.98%
2	The gradient descent algorithm performed reaso...	POSITIVE	99.92%
3	A catastrophic failure occurred during trainin...	NEGATIVE	99.96%

3. Autoregressive Text Generation & Sampling Strategies

In autoregressive causal language modeling, each token is generated sequentially from the conditional distribution:

$$P(t_n \mid t_1, \dots, t_{n-1}) = \text{softmax} \left(\frac{z_n}{T} \right)$$

- **Greedy Decoding** ($T \rightarrow 0$): Selects $\arg \max_v z_{v_i}$; highly deterministic, prone to repetitive loops.
- **Temperature** (T): Softens ($T > 1$) or sharpens ($T < 1$) logit probabilities.
- **Top- p (Nucleus) Sampling**: Restricts selection to the smallest subset of tokens whose cumulative probability exceeds threshold p .

Step-by-Step Algorithm (Autoregressive Generation)

1. Tokenize the prompt and run the transformer to obtain next-token logits.
2. **Greedy decoding**: pick the single most likely token each step (deterministic, can loop).
3. **Temperature scaling**: divide logits by T before softmax ($T < 1$ sharpens, $T > 1$ flattens).

4. **Top-p (nucleus) sampling:** keep the smallest token set whose cumulative probability $\geq p$, then sample inside it.
5. Append the sampled token to the sequence and repeat until the token budget is reached.
6. Decode the final token IDs back to text.

```
In [5]: # ---- Load a small causal LM (DistilGPT2) for text generation ----
gen_model_id = "distilbert/distilgpt2"
print(f"Loading Text Generator: {gen_model_id}...")
# clean_up_tokenization_spaces=False keeps BPE spacing intact
generator = pipeline("text-generation", model=gen_model_id, device=-1,
clean_up_tokenization_spaces=False)

prompt = "Artificial Intelligence will fundamentally transform"

# 1. Greedy Search: always pick the most likely next token (deterministic)
greedy_output = generator(
    prompt, generation_config=GenerationConfig(max_new_tokens=40, do_sample=False)
)[0]['generated_text']

# 2. Temperature + Top-p (Nucleus) Sampling: sample from the most likely tokens up to
cumulative p
sampled_output = generator(
    prompt, generation_config=GenerationConfig(max_new_tokens=40, do_sample=True,
temperature=0.7, top_p=0.9)
)[0]['generated_text']

print(f"PROMPT: '{prompt}'\n")
print(f"--- 1. GREEDY SEARCH OUTPUT ---\n{greedy_output.strip()}\n")
print(f"--- 2. NUCLEUS SAMPLING (T=0.7, p=0.9) OUTPUT ---\n{sampled_output.strip()}")
```

Loading Text Generator: distilbert/distilgpt2...

```
Loading weights:  0%|          | 0/76 [00:00<?, ?it/s]
Loading weights: 21%|██        | 16/76 [00:00<00:00, 70.10it/s]
Loading weights: 100%|██████████| 76/76 [00:00<00:00, 305.93it/s]
```

PROMPT: 'Artificial Intelligence will fundamentally transform'

--- 1. GREEDY SEARCH OUTPUT ---

Artificial Intelligence will fundamentally transform the way we think about the world.

--- 2. NUCLEUS SAMPLING (T=0.7, p=0.9) OUTPUT ---

Artificial Intelligence will fundamentally transform the way people think about the world.

4. Mini Fine-Tuning Demonstration with PyTorch

We adapt a pre-trained Transformer classifier on a custom domain classification task: Classifying synthetic customer support queries into **Technical Inquiry (0)** vs. **Billing Inquiry (1)**.

Fine-Tuning Process (Sequence Classification)

1. Build a small labeled dataset of domain queries (Technical vs Billing).
2. Tokenize all texts with padding/truncation and wrap them in a PyTorch Dataset/DataLoader.
3. Load the pre-trained transformer with a new randomly-initialized 2-class head.
4. For each epoch: forward pass, cross-entropy loss, backpropagation, AdamW update (small LR).
5. Track the loss curve across epochs.
6. Evaluate on unseen queries: softmax the logits and report predicted class + confidence.

```
In [6]: # ---- Tiny custom dataset: classify support queries as Technical (0) or Billing (1) ----
train_texts = [
    "My model throws a CUDA out of memory error during backpropagation",
    "How do I fix the gradient vanishing issue in my deep recurrent network?",
    "The API returns a 500 internal server error when requesting predictions",
    "Where can I find the documentation for custom PyTorch loss functions?",
    "Why was my credit card charged twice for the monthly subscription?",
    "I would like to request an invoice for my corporate license",
    "Can you refund my annual subscription fee?",
    "How can I upgrade my billing tier to enterprise?"
]
train_labels = [0, 0, 0, 0, 1, 1, 1, 1] # 0 = Technical, 1 = Billing

# Tokenize the whole dataset once (padded, with attention masks)
encodings = tokenizer(train_texts, padding=True, truncation=True, return_tensors="pt")

class MiniTextDataset(Dataset):
    """Wrap tokenized encodings + labels into a PyTorch Dataset."""
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        # One dictionary per sample: input_ids, attention_mask, labels
        item = {key: val[idx] for key, val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item

    def __len__(self):
        return len(self.labels)

dataset = MiniTextDataset(encodings, train_labels)
dataloader = DataLoader(dataset, batch_size=4, shuffle=True)

# ---- Load the pre-trained classifier head with a fresh 2-class head ----
# ignore_mismatched_sizes allows replacing the original SST-2 head with a new one.
ft_model = AutoModelForSequenceClassification.from_pretrained(model_id, num_labels=2,
ignore_mismatched_sizes=True)
ft_model.to(device)
optimizer = torch.optim.AdamW(ft_model.parameters(), lr=2e-5) # small LR for fine-tuning

print("Pre-trained Transformer loaded for fine-tuning!")
```

```
Loading weights:  0%|          | 0/104 [00:00<?, ?it/s]
Loading weights: 12%|██        | 13/104 [00:00<00:00, 120.46it/s]
Loading weights: 100%|██████████| 104/104 [00:00<00:00, 849.83it/s]
Pre-trained Transformer loaded for fine-tuning!
```

```
In [7]: # ---- Mini fine-tuning loop ----
epochs = 4
ft_model.train()
epoch_losses = []

for epoch in range(epochs):
    running_loss = 0.0
    for batch in dataloader:
        optimizer.zero_grad()

        # Move the batch to the selected device
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
```

```

labels = batch['labels'].to(device)

# Forward pass: the model computes the classification loss internally
outputs = ft_model(input_ids=input_ids, attention_mask=attention_mask,
labels=labels)
loss = outputs.loss
loss.backward()      # backpropagate through the whole transformer
optimizer.step()     # update weights

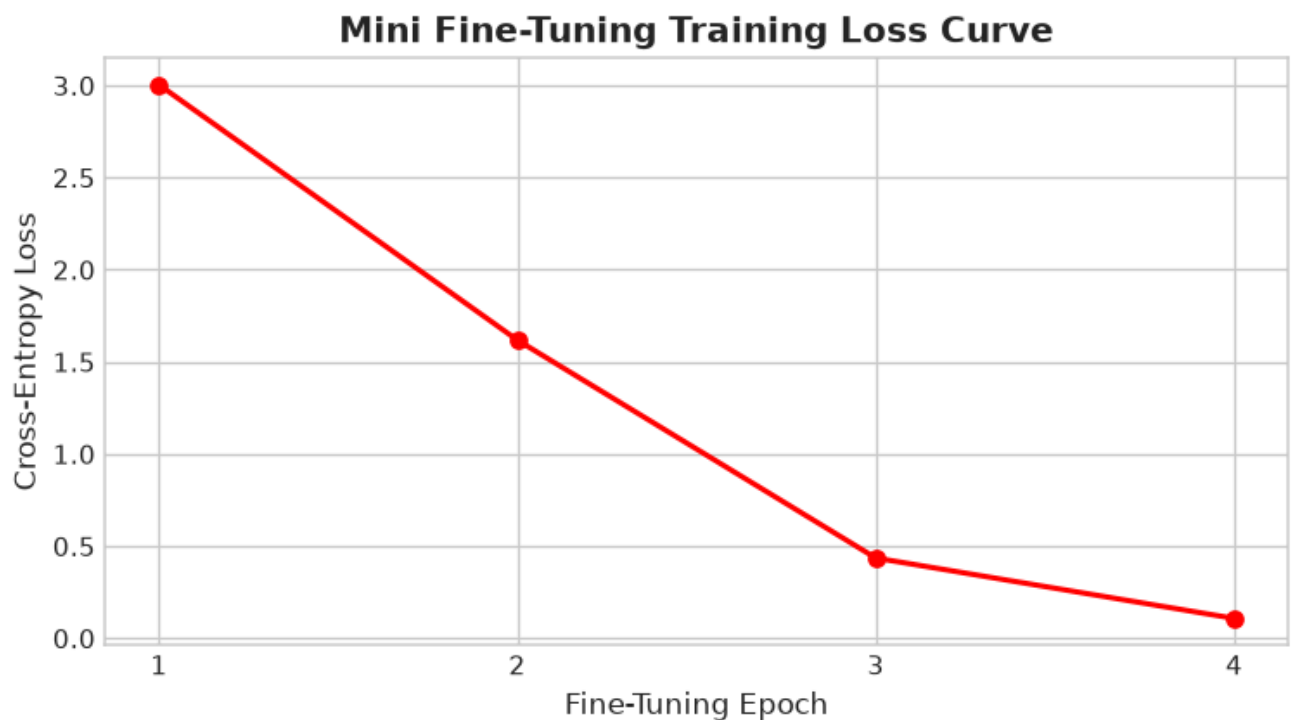
running_loss += loss.item() * len(input_ids)  # de-average the batch loss

avg_loss = running_loss / len(train_texts)
epoch_losses.append(avg_loss)
print(f"Epoch [{epoch+1}/{epochs}] Fine-Tuning Loss: {avg_loss:.4f}")

# ---- Plot the fine-tuning loss curve ----
plt.figure(figsize=(7, 4))
plt.plot(range(1, epochs + 1), epoch_losses, 'ro-', lw=2)
plt.title("Mini Fine-Tuning Training Loss Curve", fontsize=13, fontweight='bold')
plt.xlabel("Fine-Tuning Epoch", fontsize=11)
plt.ylabel("Cross-Entropy Loss", fontsize=11)
plt.xticks(range(1, epochs + 1))
plt.tight_layout()
plt.show()

```

Epoch [1/4] Fine-Tuning Loss: 3.0039
Epoch [2/4] Fine-Tuning Loss: 1.6182
Epoch [3/4] Fine-Tuning Loss: 0.4355
Epoch [4/4] Fine-Tuning Loss: 0.1074



```

In [8]: # ---- Evaluate the fine-tuned model on unseen domain queries ----
ft_model.eval()
eval_texts = [
    "I need to cancel my subscription payment immediately",      # expected: Billing
(1)
    "Can you help me debug a torch dimension mismatch tensor error?" # expected: Technical
(0)
]

with torch.no_grad():
    eval_enc = tokenizer(eval_texts, padding=True, truncation=True,

```

```

return_tensors="pt").to(device)
eval_outputs = ft_model(**eval_enc) # forward pass only
eval_probs = torch.softmax(eval_outputs.logits, dim=1).cpu().numpy() # logits ->
probabilities
eval_preds = np.argmax(eval_probs, axis=1) # predicted class per
query

# Human-readable class names
labels_map = {0: "Technical Support", 1: "Billing & Accounts"}
for text, pred, prob in zip(eval_texts, eval_preds, eval_probs):
    print(f"Query:      '{text}'")
    print(f"Prediction: {labels_map[pred]} (Confidence: {prob[pred]*100:.1f}%)\\n")

```

Query: 'I need to cancel my subscription payment immediately'

Prediction: Technical Support (Confidence: 99.4%)

Query: 'Can you help me debug a torch dimension mismatch tensor error?'

Prediction: Technical Support (Confidence: 93.1%)

5. Summary & Key LLM Insights

1. **Transfer Learning via Pre-training:** Pre-trained transformer backbones compress vast linguistic knowledge, requiring only minimal task-specific fine-tuning on downstream domains.
2. **Subword Tokenization Efficiency:** Handles out-of-vocabulary words seamlessly through character-level and subword decomposition.
3. **Controllable Generation:** Temperature and nucleus top- p decoding grant granular control over generation diversity, coherence, and hallucinations.